

Software Requirements Specification

Two practical full-stack projects for second-year students

Project	Suggested team	Recommended duration
Notes Sharing Platform	3-4 students	4-5 weeks
College Canteen Pre-Order System	3-4 students	4-5 weeks

Purpose

This document gives students a realistic, manageable specification. Each project practices core skills: responsive frontend screens, REST APIs, authentication, database design, file handling or order management, validation, and basic testing.

Suggested stack

React or plain HTML/CSS/JavaScript for the frontend; Node.js with Express for the backend;
PostgreSQL/MySQL/SQLite for the database. Students may use Bootstrap, Tailwind, or a simple custom CSS design.

Prepared for academic project work | Version 1.0 | August 2026

1. Notes Sharing Platform - Overview

1.1 Purpose

A web application where students upload, discover, download, and rate study notes organised by subject and semester. The goal is to reduce scattered sharing through messaging apps and make useful material easy to find.

1.2 In scope

- Student registration and sign-in.
- Upload PDF notes with title, subject, semester, and description.
- Browse, search, filter, preview/download notes.
- Ratings and short comments.
- Admin moderation for uploaded notes and reported content.

1.3 Users

Role	What the role can do
Student	Create an account, upload notes, search/download notes, rate and comment, report inappropriate material.
Administrator	Review reports, hide/delete unsuitable notes, manage subject and semester lists.

1.4 Assumptions and limits

- Only PDF files are accepted in the first version.
- The system is for one college; accounts use a college email or student ID.
- No live document editing, chat, or AI note generation is required.

2. Notes Sharing - Functional Requirements

ID	Requirement	Priority
NS-FR-01	The system shall allow a student to register with name, email/student ID, password, course, and semester.	Must
NS-FR-02	The system shall authenticate users and restrict upload/comment actions to signed-in students.	Must
NS-FR-03	The system shall allow a student to upload one PDF with title, subject, semester, description, and optional tags.	Must
NS-FR-04	The system shall validate file type and limit the file size (for example, 10 MB).	Must
NS-FR-05	The system shall list notes with pagination and show title, uploader, subject, semester, upload date, rating, and download count.	Must
NS-FR-06	The system shall support keyword search and filters for subject, semester, and sort order.	Must
NS-FR-07	The system shall allow a user to download a note and increment its download count.	Must
NS-FR-08	The system shall allow one rating (1-5) per student per note and allow comments from signed-in students.	Should
NS-FR-09	The system shall allow users to report a note or comment and allow an administrator to hide it.	Should
NS-FR-10	The system shall show a student's uploaded notes and permit edit/delete of their own notes.	Should

Core user flows

- Register/sign in -> browse or search -> open note details -> download or rate.
- Sign in -> upload note -> validation -> note appears as pending or published.
- Administrator -> view reports -> hide/delete material -> record moderation action.

3. Notes Sharing - Data and Interface

3.1 Main data entities

Entity	Important fields
User	id, name, email/studentId, passwordHash, course, semester, role, createdAt
Subject	id, name, course, semester
Note	id, title, description, filePath, subjectId, uploaderId, status, downloads, createdAt
Rating	id, noteId, userId, score
Comment	id, noteId, userId, text, status, createdAt
Report	id, reporterId, noteId/commentId, reason, status

3.2 Required screens

- Home/browse page with search, filters, and note cards.
- Register and sign-in pages.
- Note detail page with metadata, download button, ratings, and comments.

- Upload and My Uploads pages.
- Simple admin dashboard for reports and moderation.

3.3 Example API routes

Method	Route	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Sign in
GET	/api/notes	Search/list notes
POST	/api/notes	Upload note metadata/file
GET	/api/notes/:id/download	Download note
POST	/api/notes/:id/ratings	Rate a note
POST	/api/reports	Report content

4. Notes Sharing - Quality Requirements and Acceptance

Non-functional requirements

- Responsive layout for mobile and desktop.
- Passwords must be hashed; protected routes must reject unauthenticated users.
- Inputs must be validated on both client and server; uploaded filenames must not be trusted.
- Search results should normally appear in under 2 seconds with a small student dataset.
- Clear empty states and error messages are required.

Minimum acceptance checks

- A student can register, sign in, upload a valid PDF, and see it in search results.
- Filters show only matching notes and downloads work.
- A student cannot edit another student's note.
- An administrator can hide a reported note, after which it no longer appears in normal browsing.

Optional stretch features

- Bookmarks/favourites, email verification, PDF preview, subject-wise trending notes, file storage on a cloud service.

1. College Canteen Pre-Order System - Overview

1.1 Purpose

A web application where students view a canteen menu, place a pre-order for a pickup time, and receive an order status. Canteen staff manage the menu and process incoming orders. The goal is to reduce queue time during breaks.

1.2 In scope

- Student sign-in, menu browsing, cart, order placement, and order history.
- Available pickup slots and simulated online/COD payment choice.
- Staff dashboard to manage menu items and update order status.
- Daily order view and basic sales summary.

1.3 Users

Role	What the role can do
Student	View menu, add items to cart, select pickup time, place and cancel eligible orders, view status/history.
Canteen staff	Add/edit menu items, mark items unavailable, view new orders, change order status.
Administrator	Manage staff accounts and view daily summaries (may be combined with staff role in a small project).

1.4 Assumptions and limits

- Payment gateway integration is out of scope; payment can be marked Cash at Pickup or Simulated Paid.
- Pickup slots are fixed intervals, such as every 15 minutes.
- The first version supports one canteen and one campus.

2. Canteen Pre-Order - Functional Requirements

ID	Requirement	Priority
CP-FR-01	The system shall allow students and staff to sign in using separate roles.	Must
CP-FR-02	The system shall display menu items with name, category, price, image, availability, and estimated preparation time.	Must
CP-FR-03	The system shall allow students to add, change quantity, and remove items from a cart.	Must
CP-FR-04	The system shall calculate subtotal, optional tax/service charge, and order total before placement.	Must
CP-FR-05	The system shall allow a student to select an available pickup slot and place an order with a payment option.	Must
CP-FR-06	The system shall generate an order number and show statuses: Placed, Accepted, Preparing, Ready, Collected, Cancelled.	Must
CP-FR-07	Staff shall view orders by pickup slot/status and update the order status.	Must
CP-FR-08	Staff shall create, edit, deactivate, and set availability for menu items.	Must
CP-FR-09	Students shall see current and past orders and cancel an order only before it is accepted.	Should
CP-FR-10	The system shall display simple daily sales and popular-item summaries for staff/admin.	Should

Core user flows

- Student -> sign in -> browse menu -> cart -> pickup slot/payment -> confirmation with order number.
- Staff -> sign in -> see incoming orders -> accept -> preparing -> ready -> collected.
- Staff -> menu management -> mark unavailable item -> item cannot be added to a new cart.

3. Canteen Pre-Order - Data and Interface

3.1 Main data entities

Entity	Important fields
User	id, name, email/studentId, passwordHash, role
MenuCategory	id, name, displayOrder
MenuItem	id, name, categoryId, price, imageUrl, available, prepMinutes
PickupSlot	id, date, startTime, endTime, capacity, active
Order	id, orderNumber, studentId, slotId, status, paymentMethod, paymentStatus, total, createdAt
OrderItem	id, orderId, menuItemId, itemNameSnapshot, quantity, unitPrice
StatusLog	id, orderId, status, updatedBy, updatedAt

3.2 Required screens

- Menu page with categories and availability labels.
- Cart and checkout page with pickup-slot selector.
- Order confirmation and My Orders pages.

- Staff order board with status filters.
- Staff menu-management page and simple daily summary page.

3.3 Example API routes

Method	Route	Purpose
GET	/api/menu-items	List available menu
POST	/api/orders	Place order
GET	/api/orders/me	Student order history
PATCH	/api/orders/:id/status	Staff updates status
GET	/api/pickup-slots	Get slots/date
POST	/api/menu-items	Staff adds menu item
PATCH	/api/menu-items/:id	Staff edits availability/details

4. Canteen Pre-Order - Quality Requirements and Acceptance

Non-functional requirements

- Responsive, easy-to-read interface suitable for use on a phone during breaks.
- Only staff can change menus or order statuses; students can see only their own orders.
- Server validates prices, availability, quantities, and pickup capacity; totals are recalculated on the server.
- Order placement must not exceed the capacity of a pickup slot.
- Use clear status colours plus text labels so status is not communicated by colour alone.

Minimum acceptance checks

- A student can add available items, select a valid slot, place an order, and receive an order number.
- Unavailable items cannot be newly added to cart; server also rejects an order for an unavailable item.
- Staff can change an order from Placed to Ready, and the student sees the updated status.
- Two students cannot reserve more than the configured slot capacity.
- A student cannot view or update a different student's order.

Optional stretch features

- QR code for pickup, browser/email notifications, live order-board refresh, coupon codes, inventory counts, real payment integration after teacher approval.

5. Shared Submission Checklist

- Source code with a clear README: setup steps, environment variables, and sample accounts.
- Database schema/ER diagram and API route list.
- At least 5 screenshots covering main user flows.
- Short demo video or live presentation showing one student flow and one admin/staff flow.
- Basic test evidence: validation tests or manual test cases, including failed/unauthorised actions.
- A list of features completed, not completed, and any known limitations.